

Course Planning Document – Database Management Systems

Faculty planning notes for MCA Semester 3, DBMS course delivery

Course Objectives

The Database Management Systems course for MCA Semester 3 is designed to develop a thorough conceptual and practical understanding of how relational databases are designed, implemented, and maintained. The course objectives include enabling students to model real world data requirements using entity relationship diagrams, translate these models into relational schemas, apply normalization techniques to eliminate redundancy and anomalies, and write efficient SQL queries for data retrieval and manipulation. A further objective is to introduce students to transaction management and concurrency control, since these topics are essential for understanding how databases maintain consistency in multi-user environments.

Beyond the technical content, the course aims to cultivate an appreciation for the trade-offs involved in database design decisions, such as the balance between normalization for data integrity and denormalization for query performance, which students will encounter repeatedly in professional practice regardless of the specific database technology they eventually work with.

Learning Outcomes

On successful completion of this course, students are expected to be able to design a normalized relational database schema for a given set of requirements, identify and correct anomalies arising from poor schema design, formulate complex SQL queries involving joins, subqueries, and aggregate functions, and explain the role of transactions, concurrency control, and recovery mechanisms in maintaining database reliability. Students should also be able to reason about indexing strategies and their impact on query performance, even though a full treatment of query optimization is reserved for advanced electives.

These learning outcomes are mapped against the university's programme outcomes for the MCA curriculum, particularly those relating to the ability to design and develop solutions for complex computing problems and the ability to apply engineering knowledge to specialized domains, which in this case is data management.

Module Sequencing

The course is sequenced into five modules delivered across the semester. The first module introduces database fundamentals, the three-schema architecture, and data models, establishing vocabulary that is used throughout the remainder of the course. The second module covers entity relationship modeling in depth, including extended ER concepts such as specialization, generalization, and aggregation, since these are frequently underemphasized in introductory treatments but are important for modeling real world scenarios accurately.

The third module addresses relational algebra and SQL, deliberately placed after ER modeling so that students already understand the schemas they will be querying. The fourth module covers normalization and functional dependency theory, which is intentionally taught after students have written a reasonable

amount of SQL, since exposure to redundancy and anomalies in practice makes the motivation for normalization more concrete. The fifth and final module covers transaction management, concurrency control, and an introduction to recovery techniques, closing the course with topics that connect database theory to real world reliability concerns.

Teaching Strategy

Lectures are supplemented with live SQL demonstrations conducted on a shared database instance, allowing students to see queries executed and modified in real time rather than only viewing static examples on slides. For the normalization module, case studies drawn from realistic scenarios, such as a university enrollment system or an e-commerce order management system, are used to illustrate how unnormalized tables lead to specific, nameable anomalies, which has proven more effective than teaching normal forms through abstract definitions alone.

Peer learning is encouraged through in-class problem solving sessions where students work in pairs to design a schema for a given scenario and then present their design to the class for critique, which helps surface common misconceptions early and allows them to be addressed before the laboratory component reinforces incorrect habits.

Assessment Strategy

Assessment for this course combines two internal examinations, a set of graded assignments, laboratory evaluation, and a comprehensive end semester examination. The internal examinations are designed to test conceptual understanding through a mix of short answer and problem solving questions, particularly schema design problems and normalization exercises that require students to apply theory rather than recall definitions.

Assignments are structured to build cumulatively, with an early assignment on ER modeling feeding directly into a later assignment on schema normalization for the same scenario, so that students experience the full design pipeline from requirements to a normalized schema rather than treating each topic in isolation.

Internal Examinations

The first internal examination is scheduled after the second module and focuses on database fundamentals and ER modeling, while the second internal examination is scheduled after the fourth module and focuses on SQL and normalization, since these represent the most substantial and skill intensive portions of the syllabus. Question papers for both internals are reviewed by a second faculty member prior to administration to ensure appropriate coverage and difficulty calibration relative to the taught content.

Laboratory Activities

The DBMS laboratory is structured around a running project in which each student designs, implements, and progressively refines a database for a domain of their choosing, subject to faculty approval to ensure adequate complexity. Early laboratory sessions focus on schema creation and basic data manipulation language operations, middle sessions introduce joins, subqueries, and views, and later sessions address transaction control statements and simple concurrency scenarios simulated through multiple database sessions.

This running project approach was adopted in place of disconnected weekly exercises because it gives students a sense of ownership over their database design and allows laboratory evaluation to assess not only whether individual queries are correct but whether the overall schema remains well designed as new requirements are incorporated over the semester.